



Let's Learn C

- By Aditya Varma

Preface

Welcome to Let's Learn C

This book is written to help you learn C as a skill, not as a subject.

C is a language that shows how programs really work. It does not hide details or make assumptions for you. Because of this, learning C builds a strong foundation that makes other languages easier to understand later.

The chapters in this book are arranged carefully. Each concept is introduced only when it is needed, and every idea is supported by simple programs that explain *why* something works, not just *how* to write it.

No prior programming knowledge is assumed. You are expected only to read, practice, and stay curious. This book is meant to be read slowly, like a guide. Type the programs. Modify them. Observe what changes.

Take your time. Ask questions. Make mistakes. That is how real understanding is built.

-Aditya Varma

LICENSE & CREDITS

License:

This work is licensed under the **Creative Commons Attribution–NonCommercial–ShareAlike 4.0 (CC BY-NC-SA 4.0)** license. You may share and adapt this material for non-commercial purposes, provided proper credit is given and derivative works use the same license.

Credits:

Certain icons, illustrations, and portions of reference content are adapted from publicly available educational platforms, including the Coding X mobile application. Proper credit is given where applicable. All original rights remain with their respective creators, and this material is used solely for educational purposes.

Index

- 📖 Getting Started with C
- 📖 First C Program & Boilerplate
- 📖 Escape Sequences & Comments
- 📖 The Language Building Blocks: Tokens
- 📖 Variables and Data Types in C
- 📖 Operators in C
- 📖 Input and Output in C
- 📖 Program Flow in C
- 📖 Conditionals Statements
- 📖 Looping Statements
- 📖 Loop Control Statements
- 📖 Arrays in C
- 📖 Strings in C
- 📖 Functions in C
- 📖 Further Advance Topics

Getting Started with C

C Programming, created by Dennis Ritchie, C is a general-purpose, high-level programming language. It was created at Bell Labs in the early 1970s. One of the first operating systems to be written in C was UNIX. Even today's modern OSs have certain core parts written in C. These include Windows, Mac OS, and even Linux.

C is a perfect choice for learning to code if you are a beginner.

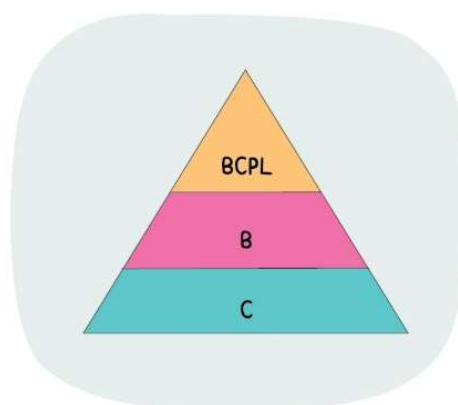
History Of C:

In the late 1960s, a high-level programming language called BCPL was created by Martin Richards, this language aimed to create a readable programming language that can be used to write Operating Systems (OS).

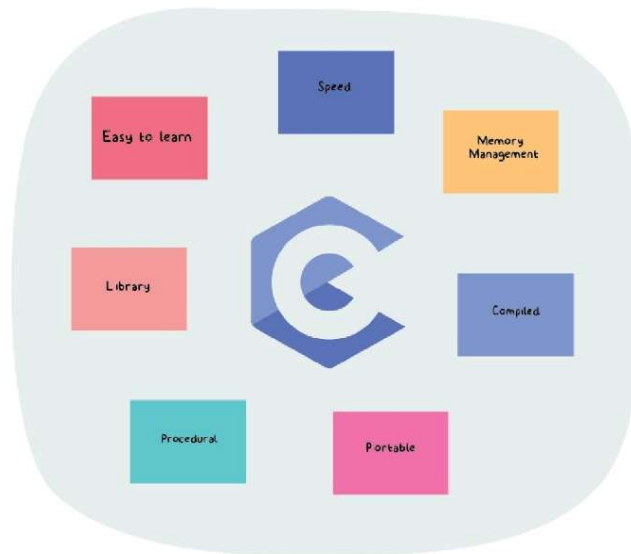
Back in those days, the programs for writing OS were not at all human-readable. This was limiting the adoption of computers for commercial purposes. And hence, this language was invented which was further improved by Ken Thompson and named "B". These languages did solve a lot of problems, but there was still an issue with memory management. Those were the old times and every byte of memory was important.

Dennis Ritchie saw this as a perfect opportunity to improve upon the B language. He tried to provide programmers with lots of memory optimization instructions along with inheriting a bulk of concepts from this existing language, B.

Since it was an improvement over the 'B' language, it was named as C Programming language. Quite convenient, eh?



Features of C:



Let's see a few reasons why you should choose C. Below are some of its amazing features.

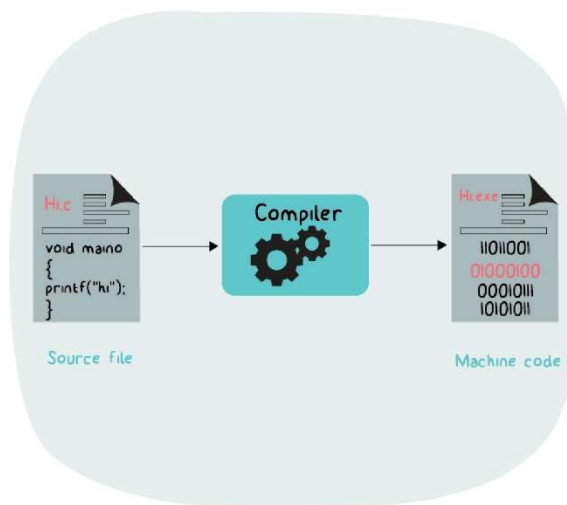
- **Easy-to-learn:** C has few keywords, a simple structure, and a clearly defined syntax. This makes it easy to catch for beginners
- **Speed:** The execution time for a C code is not the fastest among all programming languages. But its speed improves as the quantity of data to be processed increases astronomically
- **Memory Management:** C provides a variety of instructions to handle the memory allocation of its resources. If something can be achieved using lesser resources, C has the means to achieve that
- **Broad standard library:** C's bulk libraries are very portable and cross-platform compatible with UNIX, Windows, and Macintosh
- **Compiled Language:** Being a high-level language, its English-like instructions are converted into machine-level code using a software program called Compiler
- **Portable:** C can run on a wide variety of hardware platforms and has the same interface on all of those
- **Procedural Language:** Unlike an OOP language that is divided into objects and classes, C is divided into procedures or functions. This is like dividing your code into an exact set of steps to be performed.

How C Works?

C is a compiled language; it uses compiler to execute the program.

But what is a Compiler? It is a computer program that translates code written in one programming language into another language. What we as developers code is very English-based sentences. E.g. - `PRINT("SOMETHING");` This instruction needs to be converted into a machine-understandable code.

Also, a developer's code might be distributed in different files (imagine tens of documents). It is the job of the compiler to link all these files and create a final executable object that can run on the system.

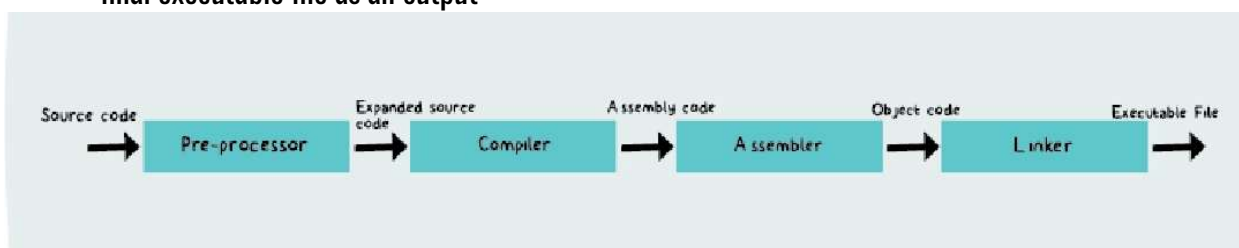


Compiled languages are converted directly into machine code that the processor can execute. As a result, they tend to be faster and more efficient to execute than interpreted languages.

Steps in Compilation:

It can be divided into 4 main tasks and they are performed in the following order,

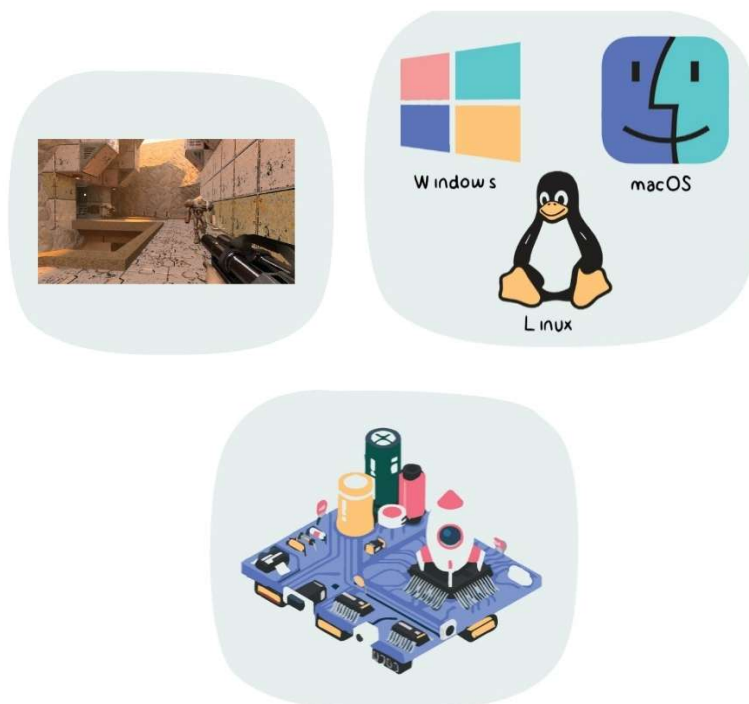
1. **Preprocessor:** This readies the code for execution by removing any comments, replacing procedure calls with actual code
2. **Compilation:** It converts pre-processed code into assembly (low-level) language
3. **Assembler:** This converts assembly language code into machine code, that is a string of 1's and 0's (binary code)
4. **Linker:** A project will have multiple files containing the source code. The linker will give one final executable file as an output



Where is C used?

Do you remember that we defined C as a general-purpose programming language? This means it is used in most places! And that's the reason why developers love C so much. C is used for,

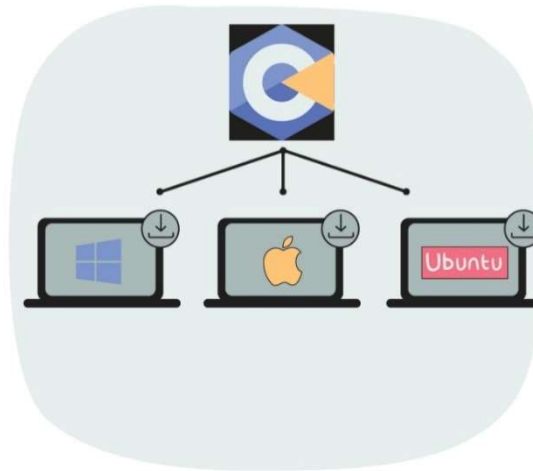
- **Desktop App Development:** C can be used to write any desktop application that you wish to develop. From a music player to photo-editing software, C has been already used. Certain parts of Mozilla Firefox Browser are written using C language.
- **Game Development:** C has a rich set of predefined graphics functions that aid in building video games. Till the early 2000s, C was heavily used for building games. The popular Unity game engine has several parts implemented using C.
- **Embedded Systems:** Due to its feature of memory management, C is the top choice for embedded systems development. It deals with microcontrollers or microprocessors, say, any machine that is controlled via inputs. C is used to create a logic that can control such devices. From rockets flying into space to a tiny smartwatch, C can handle them all!
- **Operating Systems:** We have already seen that C is used in the development of Operating Systems. In fact, it was created for this very purpose. Almost all major operating system's core kernel is written fully or partially using C. Not just operating systems, C is used to build compilers as well. That means new programming languages can also be made in C.



These are just a few of the applications noted here. C's prominent hold is on the embedded systems. That makes it a go-to language if you are thinking of a career in IoT. Of course, your options are not just limited to that. It's a 50 years old language but it surely has a future and rightly proves that - Old is Gold!

Installing C and its IDE:

To work with C on your machine, first, you will need to install the C Compiler in your system. It is unlikely that your system will be shipped with C Compiler.



Almost every C programmer uses the simplest way for getting this by using the GCC compiler. But first, would you like to know what GCC is? GCC stands for GNU Compiler Collection, it is an open-source compiler that supports the execution of multiple programming languages like C, C++, Fortran, Golang, and more.

GCC has been ported to multiple platforms and hence it helps in building C programs on a wide variety of systems. Currently, GCC's appropriate version for multiple platforms is available widely and for free.

But we are going to use something more efficient i.e., “Codeblocks”.

Installing Code::Blocks with MinGW:

Before we start writing C programs, we need a place to write and run them. For this, we will use Code::Blocks, which comes with a built-in C compiler called MinGW. Follow the steps carefully.

Step 1: Download Code::Blocks

- Open your browser
- Search for: Code::Blocks official website
- Open the website:
 - 👉 Code::Blocks
- Go to the Downloads section
- Click on “Download the binary release”

Step 2: Choose the Correct Version

You will see multiple options. Choose the one that includes MinGW.

- Look for a file named similar to:
codeblocks-20.03mingw-setup.exe

⚠ Important:

- Always choose the version with “mingw” in the name
- This version already contains the C compiler
- No need to install anything separately

Step 3: Install Code::Blocks

- Double-click the downloaded .exe file
- Click Next → I Agree → Next
- Keep default settings
- Click Install

Once installation completes:

- Click Finish

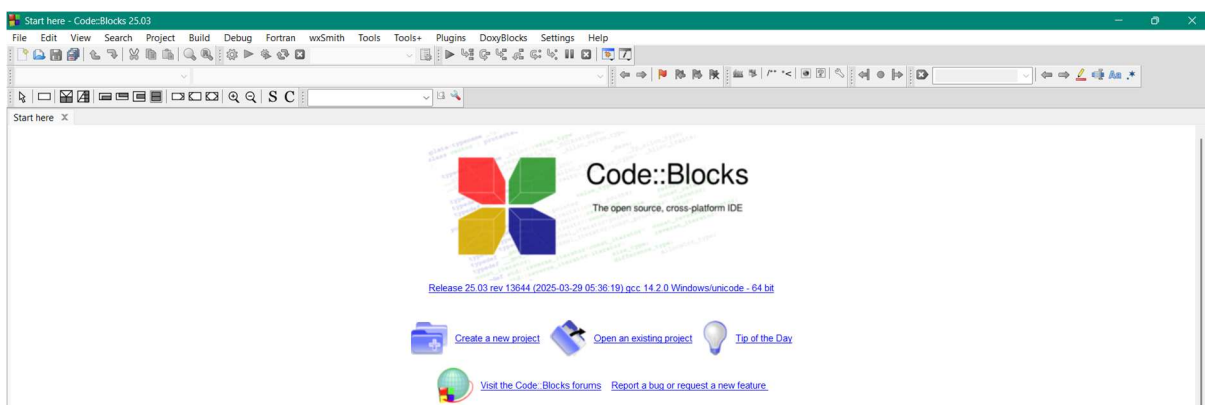
Step 4: First Launch Setup

- Open Code::Blocks
- It may ask to select a compiler

If prompted:

- Choose: GNU GCC Compiler
- Click Set as default
- Click OK

The setup is done and screen will look like this:



Creating files in Code::Blocks:

Step 1: Create Your First Project

- Click on Create a new project
- Select Console Application
- Click Go → Next
- Choose language: C
- Click Next

Step 2: Set Project Details

- Enter a Project Title (e.g., MyFirstProgram)
- Choose a folder location
- Click Next → Finish

Step 3: Write Your First Program

You will see a default file (main.c).

- Inside it, you can write your first program
- Or modify the existing code

Step 4: Build and Run the Program

- Click on Build and Run (green play button)
OR press F9
- Output will appear in a console window

At this stage, do not worry about:

- How the compiler works internally
- Complex settings
- Advanced configurations

Right now, your goal is simple:

Write → Run → Observe

First C Program & Boilerplate

After setting up the environment, the next step is simple: write something and see it work. Your first program is not about complexity. It is about starting a conversation with the computer.

Let us begin with a small program:

A screenshot of a code editor window titled 'First.c'. The code is written in C and consists of six lines. Line 1: `#include <stdio.h>`. Line 2: (empty). Line 3: `int main() {`. Line 4: `printf("Hello, World!");`. Line 5: `return 0;`. Line 6: `}`. The code is color-coded: `#include` is green, `<stdio.h>` is green, `int` is blue, `main()` is blue, `{` is blue, `printf` is blue, `"Hello, World!"` is blue, `return` is blue, `0;` is blue, and `}` is blue.

When you run this program, the output will be:

A screenshot of a terminal window. The output of the program is displayed in white text on a black background. The first line is 'Hello, World!'. The second line is 'Process returned 0 (0x0) execution time : 0.163 s'. The third line is 'Press any key to continue.'.

This is your program speaking back to you.

Understanding the Program

Let us read the program like we read a sentence.

The line `#include <stdio.h>` is written at the top. This line tells the program to bring in a set of tools before starting. These tools help in performing input and output operations. Since we are using `printf()` to display something on the screen, this line becomes necessary.

The next line, `int main()`, is where execution begins. Every C program starts from `main()`. When the program runs, the computer looks for this function and starts executing from here.

The curly braces `{ }` define a block. Everything written inside these braces belongs to the `main()` function. It is simply a way of grouping instructions together.

Inside this block, we have the line: `printf("Hello, World!");`

This is the instruction that produces output. `printf()` is used to display text on the screen, and whatever is written inside the quotes gets printed.

Finally, we see: `return 0;`

This marks the end of the program. It tells the system that the program has finished executing successfully.

Trying It Yourself:

Once you run the program, try changing the message:

```
printf("I am learning C.");
```

Or:

```
printf("This is my first program.");
```

Run it again and observe the output.

This simple action changing something and observing the result is where real learning begins.

Boilerplate Code

If you look at the program again, you will notice something interesting. Some parts of the program are not changing. No matter what you print, certain lines remain the same.

This repeated structure is called **boilerplate code**.

Boilerplate code is the basic structure required for a C program to run. It is not specific to one program. Instead, it acts as a standard starting point.

A simple version of boilerplate looks like this:

```
1  #include <stdio.h>
2
3  int main() {
4      // your code will go here
5      return 0;
6  }
```

This structure will appear in almost every C program you write.

Understanding Boilerplate Naturally

Instead of memorizing it, try to understand its role.

- The `#include` line brings in required tools.
- The `main()` function defines where execution starts.
- The curly braces hold the instructions.
- The `return 0` ends the program properly.

Everything you write will go inside this structure.

Comments & Escape Sequence

Comments in C:

When we write programs, we are not just writing for the computer. We are also writing for ourselves and for others who may read the code later. A computer only understands instructions. But humans need explanations.

This is where **comments** come in. Comments are lines in a program that are **ignored by the compiler**. They do not affect how the program runs. Instead, they help us **understand the code better**. You can think of comments as small notes written inside a program. They explain what a particular part of the code is doing, or why it was written that way.

As programs grow, they become harder to read. Without comments, even your own code can feel confusing after some time.

Comments help in:

- Explaining the purpose of the program
- Describing what a section of code does
- Making the code easier to read and maintain
- Helping others understand your logic

Even a simple program becomes clearer when comments are used properly.

Types of Comments in C:

- Single-Line Comments

A single-line comment starts with `//`. Everything written after `//` on that line is treated as a comment. This type of comment is useful for short explanations.

Example:

```
// This is a single-line comment
```

- Multi-Line Comments

A multi-line comment starts with `/*` and ends with `*/`. Everything written between these symbols is treated as a comment. This is useful when you want to write longer descriptions.

Example:

```
/*  
This is a multi-line comment.  
It can span across multiple lines.  
*/
```

Lesson Program: Comments in C**Source Code: Comments.c**

```
1  #include <stdio.h>
2
3  int main() {
4
5      // Printing a welcome message
6      printf("Welcome to C Programming!\n");
7
8      // Printing another line
9      printf("This program shows comments usage.\n");
10
11     /*
12     The following lines perform simple addition
13     and display the result
14     */
15
16     int a = 10;    // First number
17     int b = 20;    // Second number
18     int sum = a + b; // Adding numbers
19
20     printf("Sum = %d\n", sum);
21
22     return 0; // Program ends
23 }
```

Escape Sequence:

When we print something on the screen, the output appears exactly as written. But sometimes, we want more control.

We may want:

- text on a new line
- proper spacing
- special characters like quotes

This is where **escape sequences** are used. Escape sequences are **special character combinations** that begin with a backslash `\`. They tell the computer to perform a specific formatting action instead of printing normal text.

Without escape sequences, everything would print in a single continuous line, making output hard to read. This makes the output more structured and readable.

List of Escape Sequences in C

Here are the most useful escape sequences:

- `\n` → New line
- `\t` → Horizontal tab (space gap)
- `\b` → Backspace (removes previous character)
- `\r` → Carriage return (moves cursor to beginning of line)
- `\\` → Prints a backslash `\`
- `\"` → Prints double quotes `"`
- `\'` → Prints single quote `'`
- `\0` → Null character (end of string)

Lesson Program: Escape Sequence in C**Source Code:** EscapeSequence.c

```

1  #include <stdio.h>
2
3  int main() {
4      printf("Demonstrating Escape Sequences in C\n\n");
5
6      printf("1. Newline -> Hello\nWorld:\n");
7      printf("Hello\nWorld\n\n");
8
9      printf("2. Tab -> Hello\tWorld:\n");
10     printf("Hello\t World\n\n");
11
12     printf("3. Backspace -> Helloo\bWorld:\n");
13     printf("Helloo\b World\n\n");
14
15     printf("4. Carriage Return -> Hello World\rHi:\n");
16     printf("Hello World\rHioo\n\n");
17
18     printf("5. Backslash -> \\\\:n");
19     printf("\\\n\n");
20
21     printf("6. Single Quote -> \\':n");
22     printf("\\'\n\n");
23
24     printf("7. Double Quote -> \\\" :n");
25     printf("\\\"\n\n");
26
27     printf("8. Alert -> \\a:n");
28     printf("\\a\n"); // may beep if supported
29
30     return 0;
31 }

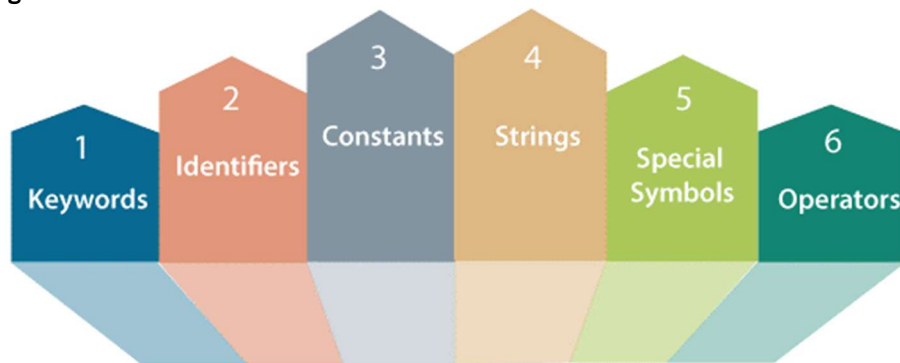
```

The Language Building Blocks: Tokens

In C programming, a token is the smallest individual unit of a program that is meaningful to the compiler. Tokens are the fundamental building blocks used to construct C programs. The C compiler analyses a program by breaking it down into these tokens to understand its structure and functionality during the compilation process.

These are primary types of tokens in C:

- Keywords
- Identifiers
- Constants & Variables
- Operators
- Data types
- String literal



Character Set:

The Character Set of any programming language indicates the different characters the program can contain. The C language identifies a character set that comprises English alphabets upper and lowercase (A to Z, as well as a to z), digits 0 to 9, and certain other symbols with a special meaning attached to them.

- Uppercase: A to Z
- Lowercase: a to z
- Digits: 0 to 9
- Special characters: ! " # \$ % & ' () * + - . : ; ` ~ = < > { } [] ^ _ \ /
- Other special characters: Blank space, tab, etc. (Compiler ignores white spaces unless they are part of string)

Keywords & Identifiers:

Every C word is classified as either a keyword or an identifier.

C Keywords:

In C, a predefined sequence of alphabets is called a keyword. Compared to human languages, programming languages have fewer keywords. To start with, C had 32 keywords, later on, few more were added in subsequent revisions of C standards. All keywords are in lowercase.

List of Keywords:

break	case	char	const	auto	short	struct	switch
double	int	else	enum	float	continue	sizeof	default
extern	for	do	goto	if	typedef	union	void
static	signed	long	register	return	unsigned	volatile	while

C Identifiers:

In C, names given to variables, functions, arrays, and other user-defined items are called identifiers. Compared to keywords, identifiers are created by the programmer to uniquely represent data and functions in a program. Unlike keywords, identifiers are not predefined; instead, they are chosen by the programmer while following certain rules.

Rules for Defining Identifiers in C,

1. Identifiers must begin with a **letter (a-z, A-Z)** or an **underscore (_)**.
2. After the first character, they can contain **letters, digits (0-9), or underscores**.
3. Identifiers **cannot be a keyword**.
4. Identifiers are **case-sensitive** (e.g., Age and age are different).
5. No special characters (like @, #, \$, %) or spaces are allowed.
6. Identifiers can be of any length, but only the first **31 characters** are significant in older C standards (modern compilers may allow more).

✓ Example Valid Identifiers:

- `sum`, `totalMarks`, `_count`, `Age1`

✗ Example Invalid Identifiers:

- `int` (keyword), `2ndValue` (starts with digit), `my-name` (special character)

Lesson Program: Keywords and Identifiers in C**Source Code: KeywordsAndIdentifiers.c**

```
1  #include <stdio.h>    // include → keyword, stdio.h → header file
2
3  int main() {          // int → keyword, main → identifier
4
5      int age = 23;      // int → keyword, age → identifier
6      float height = 5.9; // float → keyword, height → identifier
7      char grade = 'A';  // char → keyword, grade → identifier
8
9      printf("Age: %d\n", age);    // printf → identifier
10     printf("Height: %.1f\n", height);
11     printf("Grade: %c\n", grade);
12
13     return 0;          // return → keyword
14
15 }
```

Constants & Variables:

Every C program works with **data**. Data in C can be classified as either a **constant** or a **variable**.

C Constant:

In C, a constant is a fixed value that does not change during the execution of a program. Constants are used to represent values that remain the same throughout.

Constants can be of different types:

- Integer constants → 10, -25, 0
- Floating-point constants → 3.14, -0.001, 2.5e3
- Character constants → 'A', 'z', 'I'
- String constants → "Hello", "C Programming"

Symbolic constants: Data is stored as a constant and cannot be changed. Defined using `#define` or `const` keyword

C Variable:

In C, a variable is a named location in memory that stores a value which can change during program execution. Variables must be declared before use, and their type defines the kind of data they can hold.

Example: `int age = 20;` // integer variable
`float height = 5.9;` // float variable
`char grade = 'A';` // character variable

Rules for Defining Variables in C are the same as rules for Identifiers.

Lesson Program: Constants and Variables in C

Source Code: ConstantsAndVariables.c

```

1  #include <stdio.h>
2  #define PI 3.14 // symbolic constant
3
4  int main() {
5      const int days = 7; // constant
6      int age = 23; // variable
7      float height = 5.9; // variable
8      char grade = 'A'; // variable
9
10     printf("PI = %.2f\n", PI);
11     printf("Days in a week = %d\n", days);
12     printf("Age = %d, Height = %.1f, Grade = %c\n", age, height, grade);
13
14     return 0;
15 }
```


Variables And Data Types

Until now, we have written programs that display output. But real programs do more than just print messages. They **store values**, **use them**, and sometimes **change them over time**.

Think about this simple situation:

- A student has marks
- A shop has prices
- A program calculates a result

All of these require the program to **remember values**. A computer does not “remember” things the way we do. It stores everything inside its memory. This is where variables and data types comes.

Variables in C:

A variable is a **place in memory** where a value is stored. When a program runs, the computer uses its memory to hold data. Instead of remembering memory locations directly, we give them **names**. These names are called **variables**.

So, in simple terms: A variable is a named storage location used to hold data.

Each variable:

- stores a value
- has a type (what kind of data it holds)
- is stored somewhere in memory (with an address)

Different types of data require different amounts of memory and support different kinds of operations.

Variables and Memory:

You can imagine memory as a collection of boxes:

- Each box has a **name** → variable
- Each box stores a **value**
- Each box has a **type** → defines what can be stored

For example:

```
int age = 21;
```

- `int` → type (integer)
- `age` → variable name
- `21` → value stored

Naming Rules (Same as Identifiers in C):

Variable names must follow certain rules:

- Must start with a letter (a–z / A–Z) or underscore _
- Can contain letters, digits, and underscore
- Cannot start with a number
- Cannot be a keyword (int, return, etc.)
- No spaces or special symbols (@ # \$ % -)
- Case-sensitive (score and Score are different)
- Should be meaningful (totalMarks, count, price)

Examples of Valid Names:

```
int total;  
int _index;  
int value2;
```

Declaring a Variable:

Declaration means telling the compiler: “I want to create a variable of this type with this name”

```
int age;  
float salary;  
char grade;
```

At this stage:

- Memory is reserved
- But no value is stored yet

Assigning a Value:

Assignment means giving a value to an already declared variable.

```
age = 21;  
salary = 55000.0f;  
grade = 'A';
```

This updates the value stored in memory.

Initializing a Variable:

Initialization means declaring and assigning a value at the same time.

```
int year = 2025;  
float pi = 3.14159f;  
char letter = 'C';
```

This is the most common and recommended way.

Lesson Program: Variables in C**Source Code: Variables.c**

```
1  #include <stdio.h>
2
3  int main() {
4
5      // Declaration
6      int age;
7      float salary;
8      char grade;
9
10     // Assignment
11     age = 21;
12     salary = 55000.0f;
13     grade = 'A';
14
15     // Initialization (declaration + value together)
16     int year = 2025;
17     float pi = 3.14f;
18     char section = 'B';
19
20     // Using and printing variables
21     printf("Age: %d\n", age);
22     printf("Salary: %.2f\n", salary);
23     printf("Grade: %c\n", grade);
24
25     printf("Year: %d\n", year);
26     printf("Pi: %.2f\n", pi);
27     printf("Section: %c\n", section);
28
29     return 0;
30 }
```

Data Types in C:

In C, data types define the type of data a variable can store, the memory it occupies, and the range of values it can hold.

C data types are classified into:

- Primary (Fundamental) Data Types
- Derived Data Types (arrays, pointers, structures, unions)
- User-defined Data Types (typedef, enum, struct, union)

Primary Data Types in C:

Primary (fundamental) data types are the basic built-in types supported directly by the compiler. They include integers, floating-point numbers, characters, and void.

Primary Data Types with Size & Range:

Data Type	Size (bytes)	Range	Format Specifier
short int	2	-32,768 to 32,767	%hd
unsigned short int	2	0 to 65,535	%hu
unsigned int	4	0 to 4,294,967,295	%u
int	4	-2,147,483,648 to 2,147,483,647	%d
long int	4	-2,147,483,648 to 2,147,483,647	%ld
unsigned long int	4	0 to 4,294,967,295	%lu
long long int	8	-(2 ⁶³) to (2 ⁶³)-1	%lld
unsigned long long int	8	0 to 18,446,744,073,709,551,615	%llu
signed char	1	-128 to 127	%c
unsigned char	1	0 to 255	%c
float	4	1.2E-38 to 3.4E+38	%f
double	8	1.7E-308 to 1.7E+308	%lf
long double	16	3.4E-4932 to 1.1E+4932	%Lf

Note: Size and ranges may vary slightly depending on compiler and architecture (16-bit, 32-bit, 64-bit).

Lesson Program: Data Types in C**Source Code: DataTypes.c**

```

1  #include <stdio.h>
2
3  int main() {
4      char letter = 'A';           // char
5      int age = 23;                 // int
6      float pi = 3.14159;          // float
7      double big = 123456.789012345; // double
8
9      printf("char: %c\n", letter);
10     printf("int: %d\n", age);
11     printf("float: %.5f\n", pi);
12     printf("double: %.12lf\n", big);
13
14     unsigned int u_age = 40000;    // unsigned int
15     short int s_num = -30000;      // short int
16     unsigned short int us_num = 60000; // unsigned short int
17     long int population = 2000000000; // long int
18     unsigned long int u_population = 4000000000; // unsigned long int
19     unsigned char u_char = 250;    // unsigned char
20     long double huge = 3.141592653589793238L; // long double
21
22     printf("unsigned int: %u\n", u_age);
23     printf("short int: %d\n", s_num);
24     printf("unsigned short int: %u\n", us_num);
25     printf("long int: %ld\n", population);
26     printf("unsigned long int: %lu\n", u_population);
27     printf("unsigned char: %u\n", u_char);
28     printf("long double: %.18Lf\n", huge);
29
30     return 0;
31 }

```

Printing Variables in C

Once a variable stores a value, the next step is to **see and use that value**. In C, this is done using the `printf()` function. But printing is not just about displaying values. The computer must know **what kind of data** it is printing.

This is where **format specifiers** come in.

Format Specifiers:

A format specifier is a placeholder inside `printf()` that tells the program: “What type of value should be printed here” Each data type has its own format specifier.

For example: `printf("%d", n);`

Here:

- `%d` → expects an integer
- `n` → value to be printed

Common Format Specifiers

Data Type	Specifier	Example
int, short	<code>%d</code>	<code>printf("%d", n);</code>
unsigned int	<code>%u</code>	<code>printf("%u", u);</code>
long int	<code>%ld</code>	<code>printf("%ld", L);</code>
long long int	<code>%lld</code>	<code>printf("%lld", LL);</code>
float	<code>%f</code>	<code>printf("%f", f);</code>
double	<code>%lf</code>	<code>printf("%lf", d);</code>
char	<code>%c</code>	<code>printf("%c", ch);</code>
string	<code>%s</code>	<code>printf("%s", name);</code>
size_t	<code>%zu</code>	<code>printf("%zu", sizeof(int));</code>
pointer	<code>%p</code>	<code>printf("%p", ptr);</code>

Width and Precision

You can control how values appear on the screen.

- `%.2f` → prints only 2 decimal places
- `%5d` → prints number with width 5
- `%-5d` → left-align within width

These help make output **clean and structured**.

Printing Expressions

printf() can also evaluate expressions directly.

```
printf("2 + 3 = %d\n", 2 + 3);
```

The expression is calculated first, then printed.

This means you can print:

- calculations
- formulas
- results directly

Lesson Program: Format Specifiers in C

Source Code: FormatSpecifiers.c

```

1  #include <stdio.h>
2
3  int main(void) {
4
5      int n = 123;
6      long int L = 2000000000L;
7      float f = 3.14159f;
8      double d = 2.718281828;
9      char ch = 'C';
10     char name[] = "Ada";
11
12     // Printing different data types
13     printf("int: %d\n", n);
14     printf("long: %ld\n", L);
15     printf("float: %f\n", f);
16     printf("double: %lf\n", d);
17     printf("char: %c\n", ch);
18     printf("string: %s\n", name);
19
20     // Width and precision
21     printf("pi (2 decimals): %.2f\n", f);
22     printf("n right-aligned width 6: |%6d|\n", n);
23     printf("n left-aligned width 6 : |%-6d|\n", n);
24
25     // Printing expressions
26     printf("2 + 3 = %d\n", 2 + 3);
27     printf("Area 7 x 5 = %d\n", 7 * 5);
28
29     return 0;
30 }
```

Operators in C

In C, **operators** are special symbols that perform specific operations on one or more operands (variables or values) to produce a result. Operators are the backbone of expressions in C and help in performing calculations, decision-making, and data manipulation.

Types of Operators in C:

1. **Arithmetic Operators**
 - Perform basic mathematical operations.
 - Operators: `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), `%` (modulus).
 - Example (Math use):
 - `10 + 5 = 15`
 - `10 - 5 = 5`
 - `10 * 5 = 50`
 - `10 / 5 = 2`
 - `10 % 3 = 1`
2. **Relational Operators**
 - Compare two values and return true (1) or false (0).
 - Operators: `<`, `>`, `<=`, `>=`, `==`, `!=`.
 - Example (Math use):
 - `10 > 5 → true`
 - `7 == 3 → false`
3. **Logical Operators**
 - Used to combine relational expressions.
 - Operators: `&&` (logical AND), `||` (logical OR), `!` (logical NOT).
 - Example (Math use):
 - `(10 > 5) && (5 < 20) → true`
 - `(10 < 5) || (5 < 20) → true`
 - `!(10 > 5) → false`
4. **Bitwise Operators**
 - Perform operations on binary representations of integers.
 - Operators: `&` (AND), `|` (OR), `^` (XOR), `~` (NOT), `<<` (left shift), `>>` (right shift).
 - Example (Binary use):
 - `5 & 3 → 1 (0101 & 0011 = 0001)`
 - `5 | 3 → 7 (0101 | 0011 = 0111)`
5. **Assignment Operators**
 - Assign values to variables.
 - Operators: `=`, `+=`, `-=`, `*=`, `/=`, `%=`.
 - Example (Math use):
 - `a = 10`
 - `a += 5 → a = a + 5`

6. Unary Operators

- Operate on a single operand.
- Operators: `++` (increment), `--` (decrement), `!` (logical NOT), `~` (bitwise NOT).
- Example (Math use):
 - `a = 5, ++a → 6`
 - `a = 5, --a → 4`

7. Ternary (Conditional) Operator

- Shorthand for if-else condition.
- Operator: `?:`
- Example (Math use):
 - `(10 > 5) ? 1 : 0 → 1`

8. Miscellaneous Operators

- `sizeof()` → gives the size of a data type.
- `,` (comma) → separate expressions.
- `&` → address of variable.
- `*` → pointer to variable.

Precedence and Associativity of Operators

- Precedence determines which operator is evaluated first.
- Associativity determines the direction of evaluation (left-to-right or right-to-left).

Precedence	Operator	Description	Associativity
1	<code>()</code> <code>[]</code> <code>.</code> <code>→</code> <code>++</code> <code>--</code>	Parentheses or function call Brackets or array subscript Dot or Member selection operator Arrow operator Postfix increment/decrement	left to right
2	<code>++</code> <code>--</code> <code>+</code> <code>-</code> <code>!</code> <code>~</code> (type) <code>*</code> <code>&</code> <code>sizeof</code>	Prefix increment/decrement Unary plus and minus NOT operator and bitwise complement Type case Indirection or dereference operator Address of operator Determine size in bytes	right to left
3	<code>*</code> <code>/</code> <code>%</code>	Multiplication, division, and modulus	left to right
4	<code>+</code> <code>-</code>	Addition and subtraction	left to right
5	<code><<</code> <code>>></code>	Bitwise left shift and right shift	left to right
6	<code><</code> <code><=</code> <code>></code> <code>>=</code>	Relational less than and less than or equal to Relational greater than and greater than or equal to	left to right
7	<code>==</code> <code>!=</code>	Relational equal to and not equal to	left to right
8	<code>&</code>	Bitwise AND	left to right
9	<code>^</code>	Bitwise exclusive OR	left to right
10	<code> </code>	Bitwise inclusive OR	left to right
11	<code>&&</code>	Logical AND	left to right
12	<code> </code>	Logical OR	left to right
13	<code>?:</code>	Ternary operator	right to left
14	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code> <code>&=</code> <code>^=</code> <code> =</code> <code><<=</code> <code>>>=</code>	Assignment operator Addition/ subtraction assignment Multiplication/ division assignment Modulus and bitwise assignment Bitwise exclusive and inclusive OR Bitwise left shift and right shift	right to left
15		Comma operator	left to right

Lesson Program: Operators in C**Source Code: Operators.c**

```

1  #include <stdio.h>
2
3  int main() {
4      int a = 10, b = 3, result;
5      char ch = 'A';
6
7      // Arithmetic Operators
8      printf("Arithmetic: %d + %d = %d\n", a, b, a + b);
9
10     // Relational Operators
11     printf("Relational: %d > %d = %d\n", a, b, a > b);
12
13     // Logical Operators
14     printf("Logical: (%d > %d) && (%d < %d) = %d\n", a, b, a, 20, (a > b) && (a < 20));
15
16     // Bitwise Operators
17     printf("Bitwise: %d & %d = %d\n", a, b, a & b);
18
19     // Assignment Operators
20     result = a;
21     result += b;
22     printf("Assignment: result += b → %d\n", result);
23
24     // Unary Operators
25     printf("Unary: ++a = %d, --b = %d\n", ++a, --b);
26
27     // Ternary Operator
28     printf("Ternary: (a > b) ? a : b = %d\n", (a > b) ? a : b);
29
30     // Miscellaneous
31     printf("sizeof(int) = %zu\n", sizeof(int));
32     printf("Character ch = %c, ASCII = %d\n", ch, ch);
33
34     return 0;
35 }

```

C Statements & Expressions

Expressions:

An expression in C is a combination of operands (constants, variables) and operators that produces a value. Every expression has a value and a data type. Operands are what an operator acts on. Operands can be constants, variables, or other expressions.

Example Expressions:

- $6 + 4$
- $21 * (6 + d) / 20$
- $x = y + z$

Statements:

A statement is the smallest executable unit in a C program. Statements are constructed from expressions and define the actions of the program.

Types of Statements in C:

Declaration Statement

- Used to declare variables.
- Example: `int num;`

Assignment Statement

- Assigns a value to a variable.
- Example: `num = 5;`

Function Call Statement

- Calls a function to perform an operation.
- Example: `printf("Hello");`

Control Statement

- Used to control program flow.
- Example: `if (num > 0)`
- Example: `while (num < 10)`

Return Statement

- Ends a function and optionally returns a value.
- Example: `return 0;`

Expression Statement

- An expression followed by `;`.
- Example: `x++;`

Lesson Program: Statements and Expressions in C**Source Code:** StatementsAndExpressions.c

```
1  #include <stdio.h>
2
3  int main() {
4      // Declaration statement
5      int a, b, sum;
6
7      // Assignment statement
8      a = 10;
9      b = 20;
10
11     // Expression statement
12     sum = a + b;
13
14     // Function call statement
15     printf("Sum = %d\n", sum);
16
17     // Control statement
18     if (sum > 15) {
19         printf("Sum is greater than 15\n");
20     }
21
22     // Return statement
23     return 0;
24 }
```

Input and Output in C

In C programming, **Input/Output (I/O)** refers to the way a program interacts with the user.

- **Input** → Accepting data from the user (keyboard).
- **Output** → Displaying data to the user (screen).

C provides two categories of I/O functions:

1. **Formatted I/O Functions** (`printf`, `scanf`)
2. **Unformatted I/O Functions** (`getch`, `getche`, `getchar`, `gets`, `putch`, `putchar`, `puts`)

Formatted Input/Output:

Formatted I/O functions allow input/output in a specific format using format specifiers.

a. `printf()` – Output Function

Description: Used to display text, variables, and results on the screen.

Syntax: `printf("format string", variable1, variable2, ...);`

Common Format Specifiers:

- `%d` → integer
- `%f` → float (default 6 decimal places)
- `%.2f` → float (2 decimal places)
- `%c` → character
- `%s` → string

Example:

```
int a = 10;
float b = 5.75;
printf("Integer: %d, Float: %.2f", a, b);
```

Output:

Integer: 10, Float: 5.75

b. `scanf()` – Input Function

Description: Used to take input from the user and store it in variables.

Syntax: `scanf("format string", &variable1, &variable2, ...);`

(Note: `&` is used to provide the address of the variable)

Example:

```
int age;
float marks;
scanf("%d %f", &age, &marks);
printf("Age: %d, Marks: %.1f", age, marks);
```

Input:

20 88.5

Output:

Age: 20, Marks: 88.5

Lesson Program: Formatted I&O in C**Source Code: FormattedI&O.c**

```
1  #include <stdio.h>
2
3  int main() {
4      int roll;
5      float marks;
6      char grade;
7
8      // Taking input using scanf (formatted input)
9      printf("Enter Roll Number: ");
10     scanf("%d", &roll);
11
12     printf("Enter Marks: ");
13     scanf("%f", &marks);
14
15     printf("Enter Grade: ");
16     scanf(" %c", &grade); // Notice space before %c to handle newline issues
17
18     // Displaying output using printf (formatted output)
19     printf("\n--- Student Details ---\n");
20     printf("Roll Number: %d\n", roll);
21     printf("Marks: %.2f\n", marks);
22     printf("Grade: %c\n", grade);
23
24     return 0;
25 }
```

Program Flow in C

When a C program runs, it follows a **specific order of execution**. By default, the computer executes instructions **one by one, from top to bottom**. This is called **sequential execution**.

In simple programs, this path is straight. But in real programs, we often need flexibility. For example:

- Sometimes we want the program to **make decisions**
- Sometimes we want to **repeat actions**
- Sometimes we want to **change the path of execution**

This is where program flow becomes more interesting. In programming, control flow refers to the order in which individual statements, instructions, or function calls are executed. This is where control flow statements come in.

1. **Conditionals Statements:** Used when you want the program to **make choices** based on conditions (true/false).
 - if statement
 - if...else statement
 - if...else if...else ladder
 - switch statement
2. **Looping Statements:** Used when you want to repeat a block of code multiple times until a condition is met.
 - for loop
 - while loop
 - do...while loop
3. **Branching Statements:** Used when you want to change the normal flow of the program and jump to another part.
 - break → exit a loop or switch immediately
 - continue → skip current iteration and move to next
 - goto → jump to a labeled statement in code

Conditionals Statements

Conditionals (also called decision-making statements) are used in C to make programs choose a path of execution depending on whether a condition is true or false. They make a program more dynamic and logical by allowing different outputs for different situations.

Types of Conditionals in C:

1. **if Statement:** Used to execute a block of code only if the condition is true.

Syntax: `if (condition) {`
 `// code to execute if condition is true`
`}`

Example: `if (age >= 18) {`
 `printf("You can vote.\n");`
`}`

2. **if-else Statement:** Provides two-way decision making. If the condition is true → one block runs, otherwise the else block runs.

Syntax: `if (condition) {`
 `// code if condition is true`
`} else {`
 `// code if condition is false`
`}`

Example: `if (num % 2 == 0) {`
 `printf("Even\n");`
`} else {`
 `printf("Odd\n");`
`}`

3. **else-if Ladder:** Used when there are multiple conditions to check. The first true condition executes and the rest are skipped.

Syntax: `if (condition1) {`
 `// code`
`} else if (condition2) {`
 `// code`
`} else {`
 `// code if none are true`
`}`

Example: `if (marks >= 90) {`
 `printf("Grade A\n");`
`} else if (marks >= 75) {`
 `printf("Grade B\n");`
`} else {`
 `printf("Grade C or Fail\n");`
`}`

4. **Nested if Statement:** An if inside another if. Used for multiple levels of checking.

Syntax:

```
if (condition1) {
    if (condition2) {
        // code if both are true
    }
}
```

Example:

```
if (num > 0) {
    if (num % 2 == 0) {
        printf("Positive Even\n");
    }
}
```

5. **switch Statement:** Used when you need to choose one option out of many fixed cases. Works only with integral/character values.

Syntax:

```
switch (expression) {
    case value1:
        // code
        break;
    case value2:
        // code
        break;
    default:
        // code if no case matches
}
```

Example:

```
switch (day) {
    case 1:
        printf("Monday\n");
        break;
    case 2:
        printf("Tuesday\n");
        break;
    case 3:
        printf("Wednesday\n");
        break;
    default:
        printf("Invalid Day\n");
}
```

Rules for Switch Statement in C:

1. The switch expression must be of an integer or character type.
(e.g., `int`, `char`, `short`, `long`. Floating-point values are not allowed).
2. The case values must be constant expressions or literals.
(e.g., `case 1`; `case 'a'`). Variables or ranges are not allowed.
3. Duplicate case values are not allowed inside the same switch statement. Each case must be unique.
4. The break statement is optional but recommended.
 - If break is used → control exits the switch block after executing that case.
 - If break is missing → execution continues into the next case (this is called fall-through).
5. The default case is optional.
 - It executes when no other case matches.
 - It can be placed anywhere inside the switch block, but by convention, it is kept at the end.

Lesson Program: if & if-else Statement in C**Source Code:** Conditionals1.c

```

1  #include <stdio.h>
2
3  int main() {
4      int num;
5      printf("Enter a number: ");
6      scanf("%d", &num);
7
8      // Example of 'if'
9      if (num > 0) {
10         printf("The number is positive.\n");
11     }
12
13     // Example of 'if-else'
14     if (num % 2 == 0) {
15         printf("The number is Even.\n");
16     } else {
17         printf("The number is Odd.\n");
18     }
19
20     return 0;
21 }

```

Lesson Program: else-if & Nested if-else Statement in C**Source Code:** Conditionals2.c

```

1  #include <stdio.h>
2
3  int main() {
4      int marks;
5      printf("Enter your marks (0-100): ");
6      scanf("%d", &marks);
7
8      // Example of if - else if - else
9      if (marks >= 90) {
10         printf("Grade: A\n");
11     } else if (marks >= 75) {
12         printf("Grade: B\n");
13     } else if (marks >= 50) {
14         printf("Grade: C\n");
15     } else {
16         printf("Grade: Fail\n");
17     }
18
19     // Example of nested if
20     if (marks >= 50) {
21         if (marks >= 90) {
22             printf("Excellent performance!\n");
23         } else {
24             printf("You passed, keep improving.\n");
25         }
26     } else {
27         printf("You need to work harder.\n");
28     }
29
30     return 0;
31 }

```

Lesson Program: switch Statement in C**Source Code:** Conditionals3.c

```
1  #include <stdio.h>
2
3  int main() {
4      int day;
5      printf("Enter day number (1-7): ");
6      scanf("%d", &day);
7
8      switch (day) {
9          case 1:
10             printf("Sunday\n");
11             break;
12          case 2:
13             printf("Monday\n");
14             break;
15          case 3:
16             printf("Tuesday\n");
17             break;
18          case 4:
19             printf("Wednesday\n");
20             break;
21          case 5:
22             printf("Thursday\n");
23             break;
24          case 6:
25             printf("Friday\n");
26             break;
27          case 7:
28             printf("Saturday\n");
29             break;
30          default:
31             printf("Invalid day number!\n");
32      }
33
34      return 0;
35 }
```

Looping Statements

In C programming, loops are used to execute a block of code repeatedly until a given condition is true. They are also called iterative statements or repetitive control structures.

Why Loops?

1. Code reusability – no need to write the same code multiple times.
2. Efficiency – saves time, effort, and reduces errors.
3. Traversal – useful for working with data structures (arrays, strings, linked lists, etc.).
4. Automation – repeated tasks like calculations, table generation, searching, etc.

Types of Loops in C:

1. **for loop:** The for loop is best when the number of iterations is known in advance.

Syntax: `for (initialization; condition; update) {
 // code to execute`

`}`

- **Initialization** → executed once at the beginning (e.g., `int i=0;`).
- **Condition** → checked before every iteration. If false, the loop ends.
- **Update** → executed after every iteration (e.g., `i++`).

Example: `int i;
 for (i = 1; i <= 5; i++) {
 printf("Hello %d\n", i);
 }
 return 0;`

Output: `Hello 1
Hello 2
Hello 3
Hello 4
Hello 5`

2. **while Loop:**

The while loop is best when the number of iterations is not known in advance.

It is a pre-tested loop → condition is checked before execution.

Syntax: `while (condition) {
 // code to execute
}`

Example: `int i = 1;
 while (i <= 5) {
 printf("Count = %d\n", i);
 i++;
 }`

Output: `Count = 1
Count = 2
Count = 3`

```
Count = 4
```

```
Count = 5
```

3. do-while Loop:

The do-while loop is a post-tested loop → condition is checked after execution.

This guarantees that the loop executes at least once.

Syntax:

```
do {
    // code to execute
} while (condition);
```

Example:

```
int i = 1;
do {
    printf("Number = %d\n", i);
    i++;
} while (i <= 5);
```

Output:

```
Number = 1
Number = 2
Number = 3
Number = 4
Number = 5
```

4. Nested Loops:

C allows **loops inside loops**. This is useful for problems like tables, patterns, and multidimensional arrays.

Syntax:

```
for (initialization; condition; update) {
    for (initialization; condition; update) {
        // inner loop code
    }
}
```

Example (Multiplication Table):

```
int i, j;
for (i = 1; i <= 3; i++) {
    for (j = 1; j <= 3; j++) {
        printf("%d x %d = %d\n", i, j, i*j);
    }
}
```

Output (Partial):

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
2 x 1 = 2
2 x 2 = 4
```

Common Rules & Errors in Loops

Rules:

1. Always make sure the condition will eventually become false, otherwise → infinite loop.
2. Any of the three parts in `for(initialization; condition; update)` can be omitted, but semicolons must remain.
 - Example: `for(;;)` → infinite loop.
3. Variables declared in initialization are local to the loop.

Common Errors

- Missing semicolons in `for` or `do-while`.

```
do {
    ...
} while (condition) // ❌ error, needs ;
```
- Wrong condition leading to infinite loop.
- Forgetting update statement, causing infinite loop.
- Placing a semicolon immediately after loop (empty loop body).

```
for (i=0; i<5; i++); // ❌ does nothing
```

Feature	for loop	while loop	do-while loop
Condition checked	Before	Before	After
Executed at least once?	❌ No	❌ No	✅ Yes
Use case	Known iterations	Unknown iterations	Must run once

Lesson Program: for Loop Statement in C**Source Code: forLoop.c**

```

1  #include <stdio.h>
2
3  int main() {
4      int i, n;
5
6      // Using for loop to print numbers
7      for (i = 1; i <= 10; i++) {
8          printf("%d\n", i);
9      }
10
11     // Using for loop to print table
12     printf("This program prints a table of squares.\n");
13     printf("Enter number of entries in table: ");
14     scanf("%d", &n);
15
16     for (i = 1; i <= n; i++){
17         printf("%10d%10d\n", i, i * i);
18     }
19
20     return 0;
21 }

```

Lesson Program: while Loop Statement in C**Source Code: whileLoop.c**

```

1  #include <stdio.h>
2
3  int main() {
4      int n, i, sum = 0, digit;
5
6      //while loop for digit count
7      printf("Enter a number: ");
8      scanf("%d", &n);
9
10     while (n > 0) {
11         digit = n % 10;    // get last digit
12         sum += digit;      // add to sum
13         n = n / 10;        // remove last digit
14     }
15
16     printf("Sum of digits = %d\n", sum);
17
18     //while loop for table
19     printf("This program prints a table of squares.\n");
20     printf("Enter number of entries in table: ");
21     scanf("%d", &n);
22
23     i = 1;
24     while (i <= n) {
25         printf("%10d%10d\n", i, i * i);
26         i++;
27     }
28
29     return 0;
30 }

```


Lesson Program: do-while Loop Statement in C**Source Code:** dowhileLoop.c

```

1  #include <stdio.h>
2
3  int main() {
4      int digits = 0, n;
5
6      printf("Enter a nonnegative integer: ");
7      scanf("%d", &n);
8
9      do {
10         n /= 10;
11         digits++;
12     } while (n > 0);
13
14     printf("The number has %d digit(s).\n", digits);
15
16     return 0;
17 }

```

Lesson Program: Nested Loop Statement in C**Source Code:** NestedLoop.c

```

1  #include <stdio.h>
2
3  int main() {
4      int i, j;
5
6      for (i = 1; i <= 3; i++) {           // Outer loop → rows
7          for (j = 1; j <= 5; j++) {       // Inner loop → columns
8              printf("* ");
9          }
10         printf("\n"); // new line after each row
11     }
12
13
14     for (i = 1; i <= 5; i++) {
15         printf("\nMultiplication Table of %d\n", i);
16         for (j = 1; j <= 10; j++) {
17             printf("%d x %d = %d\n", i, j, i*j);
18         }
19     }
20
21     return 0;
22 }

```

Branching Statements

Branching statements in C are used to **alter the normal sequential flow** of program execution. They allow the control to **jump** from one part of the program to another.

C provides the following branching statements:

1. break Statement

Used inside loops (`for`, `while`, `do-while`) and switch statements. Immediately terminates the loop or switch and transfers control to the statement following it.

Rule:

- Only valid inside loops and switch.
- If placed outside → compiler error.

Common Error: Writing `break`; outside a loop or switch → *“break statement not within loop or switch”*.

2. continue Statement

Used only inside loops. Skips the current iteration and jumps to the next iteration of the loop.

Rule:

- Valid only inside loops.
- In `for` → control goes to update expression.
- In `while` and `do-while` → control goes back to condition checking.

Common Error: Placing `continue` outside a loop → *“continue statement not within loop”*.

3. goto Statement

Used to transfer control unconditionally to a labeled statement in the same function. Works like a jump.

Syntax: `goto label;`

```
...
label: statement;
```

Rules:

- Labels must be unique within a function.
- Control cannot jump into a loop from outside, but it can jump out.

Common Error: Overusing `goto` makes code confusing → known as “spaghetti code”.

4. return Statement

Used to terminate a function and return control to the calling function. Optionally returns value.

Rules:

- Every function must have a return if its return type is not void.
- `main()` function typically ends with `return 0;`

Common Error: Missing return in non-void function → *“control reaches end of non-void function”*.

Lesson Program: Branching Statement in C**Source Code:** BranchingStatements.c

```

1  #include <stdio.h>
2
3  int main() {
4      int i;
5
6      // Example of loop with break and continue
7      for (i = 1; i <= 10; i++) {
8          if (i == 5) {
9              printf("Skipping number %d using continue.\n", i);
10             continue; // skip when i=5
11         }
12         if (i == 8) {
13             printf("Breaking the loop at %d.\n", i);
14             break; // stop loop when i=8
15         }
16         printf("Number: %d\n", i);
17     }
18
19     // Example of goto
20     printf("\nDemonstrating goto:\n");
21     goto jump; // jump to label
22     printf("This line will be skipped.\n"); // won't execute
23
24     jump:
25     printf("We have jumped here using goto!\n");
26
27     // Example of return
28     printf("\nProgram will end here using return.\n");
29     return 0; // return from main
30 }

```

Arrays in C

In C programming, an **array** is simply a collection of items that are stored next to each other in memory. Imagine you have a row of lockers in a school corridor. Each locker has a number `[index]`, and inside it, you can keep one item. Similarly, an array gives you a series of “lockers” in computer memory where you can store values.

The important thing about arrays is that **all lockers in the row must contain items of the same type**. If you create an array for integers, all its elements must be integers; you can’t suddenly put a float or a char inside. This makes arrays consistent and predictable.

Arrays can store **primitive data types** like `int`, `float`, `char`, and `double`. They can also be used with more advanced derived types such as **structures** or **pointers**, but we’ll focus on the basics first.

Declaring an Array:

Before using an array, we must declare it. Declaring means telling the computer: *“I want a row of lockers (array) with this many spaces, and each space will hold this type of value.”*

The syntax looks like this:

```
data_type array_name[array_size];
```

For example:

```
int marks[5];
```

Here:

- `int` is the data type → every element of the array will be an integer.
- `marks` is the array’s name → just like a variable name, but it now represents a collection.
- `[5]` tells the compiler that the array has 5 elements (lockers).

Remember: array indices in C always start from 0. So, if we have `marks[5]`, the valid indices are 0, 1, 2, 3, 4. Trying to access `marks[5]` will be an **error** because there is no such locker — it’s outside the array.

Initializing an Array:

Declaring an array only sets up empty lockers. To fill them with values, we need **initialization**.

There are two main ways to do this:

1. Separately assign values after declaring:

```
int marks[5]; // Declaration
marks[0] = 60; // Initialization
marks[1] = 70;
marks[2] = 80;
marks[3] = 90;
marks[4] = 75;
```

2. Declare and initialize in one line

```
int marks[5] = {60, 70, 80, 90, 75};
```

There's also a shortcut: if we give fewer values than the size, the remaining lockers are automatically filled with 0.

Example:

```
int numbers[5] = {1, 2}; // Result: {1, 2, 0, 0, 0}
```

If we don't specify the size at all, the compiler counts the values and sets the size for us:

```
int numbers[] = {10, 20, 30, 40}; // size = 4
```

Accessing Array Elements

To access an element in the array, we use its index. Think of it as saying: *“Open locker number 3 and show me what's inside.”*

```
printf("%d", marks[2]); // prints the 3rd element, i.e., 80
```

Important rules about accessing arrays:

- The **first element** is always at index 0.
- The **last element** is always at size - 1.
- If you try to access beyond this range, your program may behave unpredictably (called **garbage values** or runtime errors).

You can also use loops to access all elements one by one:

```
for(int i = 0; i < 5; i++) {
    printf("%d ", marks[i]);
}
```

Special Notes

1. Out-of-Bounds Access

If your array has size 10 (indices 0–9) and you try to access `arr[10]`, it's an error. The program might run but give random results because it touches memory not reserved for the array.

2. Designated Initialization

C allows initializing specific lockers directly by index:

```
int arr[5] = {[2] = 212};
// Only index 2 will have 212, others will be 0.
```

```
float cost[3] = {[0] = 5000.5, [2] = 3000.0};
// Result: {5000.5, 0.0, 3000.0}
```

This is very useful when you only care about some positions in the array.

Lesson Program: Arrays in C**Source Code: Arrays1.c**

```
1  #include <stdio.h>
2
3  int main() {
4      // Declaration of an integer array
5      int numbers[5];
6
7      // Initializing array elements one by one
8      numbers[0] = 10;
9      numbers[1] = 20;
10     numbers[2] = 30;
11     numbers[3] = 40;
12     numbers[4] = 50;
13
14     // Printing all elements
15     printf("Array elements are:\n");
16     for (int i = 0; i < 5; i++) {
17         printf("numbers[%d] = %d\n", i, numbers[i]);
18     }
19
20     return 0;
21 }
```

Lesson Program: Arrays declaring in C**Source Code: Arrays2.c**

```
1  #include <stdio.h>
2
3  int main() {
4      // Declaring with initialization
5      int marks[5] = {60, 70, 80, 90, 100};
6
7      // Printing array directly using loop
8      printf("Marks stored in array:\n");
9      for (int i = 0; i < 5; i++) {
10         printf("marks[%d] = %d\n", i, marks[i]);
11     }
12
13     return 0;
14 }
```

Lesson Program: Different data types of Arrays in C**Source Code: Arrays3.c**

```

1  #include <stdio.h>
2
3  int main() {
4      // Declaration and initialization of float array
5      float prices[4] = {10.5, 20.75, 30.0, 45.99};
6
7      printf("Prices stored in float array:\n");
8      for (int i = 0; i < 4; i++) {
9          printf("prices[%d] = %.2f\n", i, prices[i]);
10     }
11
12     // Character array (like a string)
13     char vowels[5] = {'a', 'e', 'i', 'o', 'u'};
14
15     printf("Vowels stored in char array:\n");
16     for (int i = 0; i < 5; i++) {
17         printf("vowels[%d] = %c\n", i, vowels[i]);
18     }
19
20     // Double array with larger precision
21     double salary[3] = {25000.50, 30000.75, 45000.99};
22
23     printf("Salaries stored in double array:\n");
24     for (int i = 0; i < 3; i++) {
25         printf("salary[%d] = %.2lf\n", i, salary[i]);
26     }
27
28     return 0;
29 }

```

Lesson Program: Input and Output of Arrays in C**Source Code: Arrays4.c**

```

1  #include <stdio.h>
2
3  int main() {
4      int marks[5];    // Array of size 5
5      int i;
6
7      // Taking input
8      printf("Enter marks of 5 students:\n");
9      for (i = 0; i < 5; i++) {
10         printf("Enter mark for student %d: ", i + 1);
11         scanf("%d", &marks[i]);    // input for array element
12     }
13
14     // Displaying output
15     printf("\nMarks of 5 students are:\n");
16     for (i = 0; i < 5; i++) {
17         printf("marks[%d] = %d\n", i, marks[i]);
18     }
19
20     return 0;
21 }

```


Multidimensional Arrays in C

A **multidimensional array** is simply an **array of arrays**.

- In **1-D arrays**, data is stored in a single row (like a list).
- In **2-D arrays**, data is stored in rows and columns (like a table or matrix).
- In **3-D arrays**, data is stored as a “stack of tables” (imagine multiple 2D tables stacked one after another).

So basically:

- **1-D** → Line of data (like student marks).
- **2-D** → Table of data (like a chessboard, timetable, or matrix).
- **3-D** → Stack of tables (like pages of an Excel sheet).

Two-Dimensional Array (2D Array)

A 2D array is like a **grid of rows and columns**. It can be defined as an array of arrays.

Example:

```
int matrix[4][3];
```

Here:

- 4 → number of rows
- 3 → number of columns
- So it creates a table with 4 rows and 3 columns (total 12 elements).

Initialization of 2D Arrays

There are two common ways:

1. Row by Row

```
int matrix[4][3] = {
    {10, 20, 30},
    {40, 50, 60},
    {70, 80, 90},
    {100, 110, 120}
};
```

2. All in one line

```
int matrix[4][3] = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120};
```

Accessing Elements in 2D Array

We use **two indices**: one for row and one for column.

```
matrix[0][2] // Row 0, Column 2 → gives 30
matrix[2][1] // Row 2, Column 1 → gives 80
```

Indices always start from **0**.

So for a 4x3 matrix:

- Rows → 0,1,2,3
- Columns → 0,1,2

Three-Dimensional Array (3D Array)

If a 2D array is like a table, a **3D array** is like a **stack of tables**.

Declaration:

```
int box[10][20][30];
```

This means:

- 10 tables
- Each table has 20 rows
- Each row has 30 columns

So total elements = $10 \times 20 \times 30 = 6000$.

Visualization

- 1D array → a row (line of data)
- 2D array → a table (rows × columns)
- 3D array → a stack of tables

Loops for Multidimensional Arrays

To work with multidimensional arrays, we use **nested loops**.

- For 2D array → 2 loops (row loop + column loop)
- For 3D array → 3 loops (layer loop + row loop + column loop)

Lesson Program: Multidimensional Arrays in C

Source Code: MultiDimensionArrays1.c

```

1  #include <stdio.h>
2
3  int main() {
4      // Declaration + Initialization
5      int matrix[2][3] = {
6          {10, 20, 30},
7          {40, 50, 60}
8      };
9
10     // Printing initialized values
11     printf("Matrix elements are:\n");
12     printf("%d %d %d\n", matrix[0][0], matrix[0][1], matrix[0][2]);
13     printf("%d %d %d\n", matrix[1][0], matrix[1][1], matrix[1][2]);
14
15     return 0;
16 }
```

Lesson Program: Accessing Multidimensional Arrays in C**Source Code:** MultiDimensionArrays2.c

```

1  #include <stdio.h>
2
3  int main() {
4      int matrix[2][3] = {
5          {1, 2, 3},
6          {4, 5, 6}
7      };
8
9      printf("Printing 2D array using loops:\n");
10     for (int i = 0; i < 2; i++) {
11         for (int j = 0; j < 3; j++) {
12             printf("%d ", matrix[i][j]); // print element at row i, col j
13         }
14         printf("\n"); // move to next row
15     }
16
17     return 0;
18 }

```

Lesson Program: Input & Output of Multidimensional Arrays in C**Source Code:** MultiDimensionArrays3.c

```

1  #include <stdio.h>
2
3  int main() {
4      int matrix[2][2]; // 2x2 array
5      int i, j;
6
7      // Input
8      printf("Enter 4 numbers for a 2x2 matrix:\n");
9      for (i = 0; i < 2; i++) {
10         for (j = 0; j < 2; j++) {
11             printf("Enter element at [%d][%d]: ", i, j);
12             scanf("%d", &matrix[i][j]);
13         }
14     }
15
16     // Output
17     printf("\nThe matrix is:\n");
18     for (i = 0; i < 2; i++) {
19         for (j = 0; j < 2; j++) {
20             printf("%d ", matrix[i][j]);
21         }
22         printf("\n");
23     }
24
25     return 0;
26 }

```

Strings in C

A **string** in C is defined as a **one-dimensional array of characters**. The key difference between a **character array** and a **string** is that a string is **terminated by a special null character '\0'**. Each character in the array occupies **1 byte** of memory, and the last character must always be '\0' to indicate the end of the string. Without the null character, the compiler cannot identify where the string ends.

Example:

```
char str[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

Here, "Hello" takes 5 characters, and the last index stores '\0'

Declaration of Strings:

Strings in C can be declared in two ways:

(a) By Character Array:

```
char ch[5] = {'j', 'a', 'v', 'a', '\0'};
```

(b) By String Literal:

```
char ch[] = "java";
```

Note: When we use a string literal, the compiler automatically appends '\0' at the end.

Difference Between Char Array & String Literal:

- In a **char array**, we must explicitly add the '\0'.
- In a **string literal**, the compiler automatically adds it.
- String literals **cannot be reassigned** to another set of characters directly, but in a character array we can modify the contents.

Lesson Program: Strings in C**Source Code:** Strings1.c

```
1  #include <stdio.h>
2
3  int main() {
4      char ch1[] = {'H','e','l','l','o','\0'};
5      char ch2[] = "Hello";
6
7      printf("String 1 = %s\n", ch1);
8      printf("String 2 = %s\n", ch2);
9
10     return 0;
11 }
```

Lesson Program: Input & Output Strings in C**Source Code:** Strings2.c

```
1  #include <stdio.h>
2
3  int main() {
4      char name[20];
5
6      printf("Enter your name: ");
7      scanf("%s", name); // Will stop at space
8
9      printf("Hello %s!\n", name);
10
11     return 0;
12 }
```

Lesson Program: Traversing Strings in C**Source Code:** Strings3.c

```
1  #include <stdio.h>
2
3  int main() {
4      char ch[] = "Aditya";
5      int i = 0;
6
7      // Method 1: using length
8      while (i < 6) {
9          printf("%c ", ch[i]);
10         i++;
11     }
12
13     printf("\n");
14
15     // Method 2: using null character
16     i = 0;
17     while (ch[i] != '\0') {
18         printf("%c ", ch[i]);
19         i++;
20     }
21
22     return 0;
23 }
```

Lesson Program: Strings Functions and uses in C**Source Code: Strings4.c**

```

1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char str1[20] = "Hello";
6      char str2[20];
7
8      // strlen() - gives length of string (excluding '\0')
9      printf("Length of str1 = %lu\n", strlen(str1));
10
11     // strcpy() - copies one string to another
12     strcpy(str2, str1);
13     printf("After strcpy, str2 = %s\n", str2);
14
15     // strncpy() - copies only first n characters
16     strncpy(str2, "World", 3);
17     str2[3] = '\0'; // must manually add null character
18     printf("After strncpy (3 chars), str2 = %s\n", str2);
19
20     // strcat() - joins two strings
21     char str3[40] = "Good ";
22     strcat(str3, "Morning");
23     printf("After strcat, str3 = %s\n", str3);
24
25     // strncat() - joins only n characters from second string
26     char str4[40] = "C ";
27     strncat(str4, "ProgrammingLanguage", 6);
28     printf("After strncat(6), str4 = %s\n", str4);
29
30     // strcmp() - compares two strings
31     printf("strcmp(\"abc\", \"abd\") = %d\n", strcmp("abc", "abd")); // -1
32     printf("strcmp(\"abc\", \"abc\") = %d\n", strcmp("abc", "abc")); // 0
33
34     // strncmp() - compares only first n characters
35     printf("strncmp(\"abcdef\", \"abcxyz\", 3) = %d\n", strncmp("abcdef", "abcxyz", 3));
36
37     return 0;
38 }

```

Lesson Program: Strings Functions and uses in C**Source Code:** Strings5.c

```

1  #include <stdio.h>
2  #include <ctype.h> // for character testing functions
3
4  int main() {
5      char str[100];
6      int i;
7
8      printf("Enter any string: ");
9      gets(str); // use fgets in modern compilers
10
11     printf("\nAnalyzing the string:\n");
12
13     for(i = 0; str[i] != '\0'; i++) {
14         if(islower(str[i]))
15             printf("%c' is a lowercase letter.\n", str[i]);
16         else if(isupper(str[i]))
17             printf("%c' is an uppercase letter.\n", str[i]);
18         else if(isdigit(str[i]))
19             printf("%c' is a digit.\n", str[i]);
20         else if(isalnum(str[i]))
21             printf("%c' is alphanumeric.\n", str[i]);
22         else if(isspace(str[i]))
23             printf("%c' is a whitespace character.\n", str[i]);
24         else if(isprint(str[i]))
25             printf("%c' is printable.\n", str[i]);
26         else if(ispunct(str[i]))
27             printf("%c' is a punctuation mark.\n", str[i]);
28         else
29             printf("%c' is a control or special character.\n", str[i]);
30     }
31
32     return 0;
33 }

```


Functions in C

In C, a function is a block of code that performs a specific task. It helps us to divide a large program into smaller, manageable parts each function performs a certain job.

You can think of a function like a machine:

- ➔ You give it input (arguments)
- ➔ It does some work (statements inside function)
- ➔ It gives output (return value)

Why use Functions?

Advantages:

1. Avoids rewriting the same code again and again.
2. Makes a program easier to understand and debug.
3. Reduces complexity by dividing a large program into smaller blocks.
4. We can call a function multiple time.
5. Helps in teamwork different people can work on different functions.
6. Promotes **reusability** of code.

Aspects of a Function

There are **3 main parts** of a C function:

1. Function Declaration (Prototype)

It tells the compiler about the function's name, return type, and parameters.

Syntax:

```
return_type function_name(argument_list);
```

2. Function Definition

This is the actual body of the function — what it does.

Syntax:

```
return_type function_name(argument_list) {
    // function body
}
```

3. Function Call

This is where the function is executed (used).

Syntax:

```
function_name(arguments);
```

Lesson Program: Functions in C**Source Code: Functions1.c**

```

1  #include <stdio.h>
2
3  // Function Declaration
4  void greet();
5
6  int main() {
7      greet();    // Function Call
8      return 0;
9  }
10
11 // Function Definition
12 void greet() {
13     printf("Hello! Welcome to C programming.\n");
14 }

```

Explanation:

- Function greet() is declared before main().
- When called, it executes its body and prints the message.

Types of Functions (Based on Parameters & Return Type)

Type	Example	Description
No arguments, No return value	<code>void show()</code>	Performs action, returns nothing
Arguments, No return value	<code>void add(int a, int b)</code>	Takes input but no return
No arguments, With return value	<code>int roll()</code>	Returns something without input
Arguments, With return value	<code>int sum(int a, int b)</code>	Takes input and returns output

Passing Data to Functions

When we pass data to a function, it can be done in two ways:

1. Call by Value:

Here, a copy of the variable is passed to the function. Changes made inside the function do not affect the original value.

Lesson Program: Call by Value Functions in C

Source Code: Functions2.c

```

1  #include <stdio.h>
2
3  void add(int a, int b) {
4      a = a + b;
5      printf("Sum inside function: %d\n", a);
6  }
7
8  int main() {
9      int x = 5, y = 10;
10     add(x, y);
11     printf("Values after function: x = %d, y = %d\n", x, y);
12     return 0;
13 }
```

2. Call by Reference:

Here, we pass the address (reference) of variables. Changes made inside the function affect the original values.

Lesson Program: Call by Reference Functions in C

Source Code: Functions3.c

```

1  #include <stdio.h>
2
3  void swap(int *a, int *b) {    // using pointers
4      int temp = *a;
5      *a = *b;
6      *b = temp;
7  }
8
9  int main() {
10     int x = 10, y = 20;
11     printf("Before swap: x = %d, y = %d\n", x, y);
12     swap(&x, &y);    // passing addresses
13     printf("After swap: x = %d, y = %d\n", x, y);
14     return 0;
15 }
```

Lesson Program: No return and arguments Functions in C**Source Code:** Functions4.c

```

1  #include <stdio.h>
2
3  // Function Declaration
4  void findSquare();
5
6  int main() {
7      // Function Call
8      findSquare();
9      return 0;
10 }
11
12 // Function Definition
13 void findSquare() {
14     int num, square;
15     printf("Enter a number: ");
16     scanf("%d", &num);
17
18     square = num * num;
19     printf("Square of %d is %d\n", num, square);
20 }

```

Lesson Program: Return type and argument Functions in C**Source Code:** Functions5.c

```

1  #include <stdio.h>
2
3  // Function Declaration
4  int findMax(int a, int b);
5
6  int main() {
7      int x, y, max;
8      printf("Enter two numbers: ");
9      scanf("%d %d", &x, &y);
10
11     // Function Call and storing return value
12     max = findMax(x, y);
13     printf("The maximum number is: %d\n", max);
14
15     return 0;
16 }
17
18 // Function Definition
19 int findMax(int a, int b) {
20     if(a > b)
21         return a;    // return larger number
22     else
23         return b;
24 }

```

Recursion in C

Until now, we have seen functions calling other functions. But C allows something even more interesting: A function can call **itself**. This concept is known as **recursion**.

Recursion is a programming technique in which a function solves a problem by repeatedly calling itself until a stopping condition is reached.

In simple words: A recursive function is a function that calls itself. At first, this may seem strange. Why would a function keep calling itself? The answer is that many problems can be broken down into **smaller versions of the same problem**. Recursion allows us to solve such problems elegantly and naturally.

The Two Parts of Every Recursive Function

Every recursive function must contain two important parts:

Base Case - The condition that stops further recursive calls. Without a base case, the function would continue calling itself forever.

Recursive Case - The part where the function calls itself with a smaller or simpler problem. This gradually moves the function toward the base case.

How Recursion Works

Suppose we want to count down from 5 to 1.

A recursive function would:

```
print(5)
print(4)
print(3)
print(2)
print(1)
stop
```

Instead of writing all those statements manually, the function repeatedly calls itself with a smaller value. You can think of recursion as standing in front of a mirror that reflects another mirror.

Each reflection creates another reflection, but eventually the reflections become too small to continue. The base case acts as that stopping point.

Lesson Program: Recursion Functions in C**Source Code: Functions6.c**

```
1  #include <stdio.h>
2
3  void countdown(int n) {
4      if (n == 0) {          // Base Case
5          return;
6      }
7
8      printf("%d\n", n);
9      countdown(n - 1);    // Recursive Call
10 }
11
12 int main() {
13     countdown(5);
14
15     return 0;
16 }
```

Lesson Program: Recursion Functions in C**Source Code: Functions7.c**

```
1  #include <stdio.h>
2
3  int factorial(int n) {
4      if (n == 1) {          // Base Case
5          return 1;
6      }
7
8      return n * factorial(n - 1);
9  }
10
11 int main() {
12     int result = factorial(5);
13     printf("Factorial = %d\n", result);
14
15     return 0;
16 }
```

Where Is Recursion Used?

Recursion is commonly used in:

- Mathematical calculations (factorial, Fibonacci)
- Searching and sorting algorithms
- File and folder traversal
- Tree and graph data structures
- Divide-and-conquer algorithms

Many advanced computer science concepts rely heavily on recursion.

Recursion vs Loops

Both recursion and loops are used for repetition.

Loop	Recursion
Uses for, while, do-while	Uses function calls
Usually easier for simple repetition	Often cleaner for recursive problems
Uses less memory	Uses more memory due to function calls
Faster in many cases	Can be more intuitive for certain problems

Things to Remember

- Every recursive function must have a **base case**
- The recursive call should move closer to the base case
- Missing a base case can cause infinite recursion
- Recursion is powerful but should be used where it makes the solution clearer

Further Advance Topics in C

Throughout this book, we focused on the core building blocks of C. You learned how to:

- Store data using variables
- Take input and display output
- Perform calculations
- Make decisions
- Repeat tasks using loops
- Work with arrays and strings
- Organize code using functions

With these skills, you can already build many useful programs.

However, C has much more to offer. As programs become larger and more complex, developers need better ways to organize data, work with memory, interact with files, and build reusable code.

The topics in this chapter are commonly learned after mastering the fundamentals. They open the door to system programming, embedded development, operating systems, game engines, and many other advanced areas.

Let us take a brief look at some of these topics.

Structures:

Structures allow us to group multiple pieces of related information into a single unit. Instead of storing a student's name, age, and marks in separate variables, a structure lets us keep them together.

Example:

```

1  #include <stdio.h>
2
3  struct Student {
4      char name[20];
5      int age;
6      float marks;
7  };
8
9  int main() {
10
11     struct Student s1 = {"Aditya", 23, 89.5};
12
13     printf("Name: %s\n", s1.name);
14     printf("Age: %d\n", s1.age);
15     printf("Marks: %.1f\n", s1.marks);
16
17     return 0;
18 }
```

Structures are widely used in:

- Student management systems
- Employee records
- Banking software
- Real-world data modelling

Pointers:

A pointer is a variable that stores the address of another variable. Pointers are one of the most powerful features of C and are heavily used in advanced programming.

Example:

```

1  #include <stdio.h>
2
3  int main() {
4      int age = 23;
5      int *ptr = &age;    // ptr stores address of age
6
7      printf("Value of age: %d\n", age);
8      printf("Address of age: %p\n", (void*)&age);
9      printf("Value through pointer: %d\n", *ptr);
10
11     return 0;
12 }
```

Pointers are used in:

- Dynamic memory allocation
- Data structures
- Operating systems
- Embedded systems

Enumerations (Enums)

Enums allow us to create meaningful names for fixed values. Instead of remembering numbers, we can use readable labels.

Example:

```

1  #include <stdio.h>
2
3  enum Day {
4      MONDAY,
5      TUESDAY,
6      WEDNESDAY
7  };
8
9  int main() {
10     enum Day today = TUESDAY;
11     printf("Today = %d\n", today);
12
13     return 0;
14 }
```

Enums improve readability and make programs easier to understand.

File Handling:

Until now, all data disappeared when the program ended. File handling allows data to be saved permanently.

Example:

```

1  #include <stdio.h>
2
3  int main() {
4      FILE *file = fopen("notes.txt", "w");
5
6      fprintf(file, "Learning C Programming");
7      fclose(file);
8      printf("Data written to file.");
9
10     return 0;
11 }
```

File handling is used for:

- Saving records
- Storing reports
- Logging information
- Managing databases

Libraries and Header Files

Libraries provide ready-made functions written by other programmers. Instead of creating everything from scratch, we can use these functions directly.

Example:

```

1  #include <stdio.h>
2  #include <math.h>
3
4  int main() {
5      double result = sqrt(25);
6
7      printf("Square Root = %.0lf\n", result);
8
9      return 0;
10 }
```

Common libraries include:

- `stdio.h` → Input and output
- `math.h` → Mathematical functions
- `string.h` → String operations
- `stdlib.h` → Utility functions

Libraries save time and make development faster.

Dynamic Memory Allocation

Normally, memory size is decided before a program runs. Dynamic memory allocation allows memory to be created while the program is running.

Example:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      int *ptr;
6      ptr = (int*) malloc(sizeof(int));
7      *ptr = 100;
8
9      printf("Value = %d\n", *ptr);
10     free(ptr);
11
12     return 0;
13 }
```

This concept is essential for large applications and advanced data structures.

Preprocessor Directives

Before a program is compiled, the preprocessor performs certain tasks. We have already used one: `#include <stdio.h>` Another common directive is: `#define PI 3.14159`

Preprocessor directives help configure programs before compilation begins.

Closing Note

The goal of this book was not to teach every feature of C. The goal was to build a strong foundation.

Once the fundamentals become second nature, advanced topics stop feeling difficult and start feeling like natural extensions of ideas you already understand. And that is where the real journey begins.



Thank You!!!